

ASSESSMENT NOTIFICATION

Task Overview

Course	Software Engineering Year 12	Assessment Task Number	3
Date of Task	Term 3, Week 1, Friday 24th of July		
Weighting	30%		
Syllabus Outcome Codes	SE-12-01, SE-12-02, SE-12-03, SE-12-04, SE-12-06, SE-12-07, SE-12-08, SE-12-09		

Task Details

Type of task	Individual Project
Content/Topics	Software Engineering Project
Task Description	You are to develop a software engineering project under the Software Engineering Project unit. This project provides an opportunity for you to demonstrate your mastery and application of subject knowledge and skills in relation to the outcomes of the course over an extended period of time. In this unit you will develop the project through identifying and defining requirements and solutions, researching, planning and documenting the project management and software engineering solutions, producing and implementing solutions, and testing and evaluating code and solutions. The Software Engineering Project unit integrates content from the focus areas Secure Software Architecture, Programming for the Web, and Software Automation. Completion of this assessment task will require you to apply your learning from the focus areas and prior learning to develop your project. You will need to use Python 3.x to implement your solution, along with any necessary external libraries and frameworks, while JavaScript (with/without frameworks) will be needed for the front-end. You will have the opportunity to demonstrate your project management skills, collaborate on problems and develop team and communication skills, which are highly valued in the industry. You will document the process of development in the provided project template folio, wherein you will provide evidence that the project is all your own work. The following parts will be delivered as the overall project task: Part A – Software engineering solution documented in the project folio Under the supervision of your teacher you will select a software development approach to produce a software engineering solution. The software engineering solution should: • address an identified need and/or opportunity • have programming code that is functional • use language-specific tools and resources • safely and securely store user data • have optimised performance • provide a functional and intuitive user experience. • Include automated testing (covering the entirety of the code), thus fulfilling the Automation requirements of the curriculum. You will develop the following project documentation to support the planning and development of your software solution: • Problem Definition • Functional and non-functional requirements • Storyboard (UML Activity Diagram) • Financial feasibility • Dataflow Diagram (DFD)

	• Data dictionary & ER diagram for your database • UML Class diagram • Project Diary • Gantt chart • Algorithm (either pseudocode OR flowchart) of the software solution • Testing and debugging documentation (what you used, results, test values, boundary conditions, consequent modifications), comparing actual output with expected output * IMPORTANT: Your teacher should be able to run the program locally on their own laptop. All dependencies should be indicated via a requirements.txt file. Accompanying this document, you should have access to a Folio Template document to assist you with the written part of your final Project Deliverable. Part B – Working Software Delivery: The second part is the delivery of a working project, addressing one of the suggested projects. While the project may be based upon the suggested project, you are free to modify/extend your project to enhance the final product (both from the front and back ends of the codebase). It is important to consult with your teacher at regular points in your project: initiation, coding, testing and delivery. All the parts of your project deliverables must be entirely your own work. Project Option Details:	
	Project 1: Secure Document Management & Approval System Project Overview Students develop a secure, role-based web application that allows users to upload, manage, and approve documents. The system emphasises access control, secure file handling, auditability, and automation.	
	Front-End Requirements (React , Vue or frameless JavaScript) • Secure login and logout interface • Role-based UI: ○ Standard user: upload & view documents ○ Reviewer: approve/reject documents ○ Admin: manage users • Document list with metadata: ○ Filename, owner, status, upload date, approve/reject data, etc. • Approval workflow interface • Error and validation feedback (client-side)	
	Back-End Requirements • REST API (Flask / FastAPI) • Secure authentication & authorisation: ○ Role-based access control • File system integration: ○ Store files outside public directory ○ Validate file type and size ○ Encrypt files as needed • Document approval state machine • Audit logging of actions: ○ Upload ○ Approval/Rejection ○ Automated reporting runs, etc • Secure session or token handling	
	Automation Requirements	

	• Automated file scanning or validation step • Scheduled cleanup of expired or rejected files • Automatic email or dashboard notifications (simulated)	
	Secure Software Architecture Focus • Separation of concerns (UI, API, file storage) • Least privilege access • Input validation and error handling • Secure file handling patterns	
	Project 2: Secure API-Driven Smart Services Dashboard Project Overview Students design a web dashboard that aggregates data from external APIs and internal services, focusing on secure API design, validation, rate-limiting concepts, and automation workflows .	
	Front-End Requirements (React or Vue) • Dashboard with multiple data panels • Dynamic charts or tables • Secure login • User-controlled refresh intervals • Error handling for API failures	
	Back-End Requirements • REST API gateway service • Secure API consumption: ○ Validate external API responses ○ Handle timeouts and failures • Internal API endpoints with: ○ Authentication checks ○ Input validation • Caching layer (conceptual or basic) • Logging of API usage	
	<u>Submission</u> You need to download a zip of your repository and submit this in Schoolbox in the Assessment 3 link in our Software Engineering course, along with any additional files, such as your Project Documentation.	
Equipment	Github Codespace/VS Code (recommended), Flask, Sqlite3, Python, Bash commands, Spreadsheet/Project Management Software, Testing Software (eg Pylint, Accessibility Checker, Pageseed), Internet, Word processor. Project Template	
Preparation	To prepare for this task, students should: • Familiarise with provided resources at the end of this document • Create a repository in git (at the very start of your project work) and share with your teacher. The repository should contain the code and the documentation/report, both of which will be updated as the project progresses. • Install necessary software extensions in VS Code / Codespace	

Marking Criteria	Marking rubric can be found on the next page. Outstanding: You clearly explain your problem, plan realistically, build a working solution, test it properly, improve your code and explain your decisions. Highly Developed: You meet most requirements and your project works, but explanations or testing may lack depth. Competent: You complete parts of the task, but planning, testing or documentation is incomplete. Developing: You attempt the task but your solution is incomplete or poorly explained. Experiencing Difficulty: You deliver nothing.
Feedback	Feedback on this task will include a mark and comments, which will be accessed on Schoolbox. Students must check that their mark has been correctly entered on Schoolbox.
Resources	Project Documentation Template (copy in the appendix of this document and also in Schoolbox) Project Management Software Engineering Course Specifications Accessibility/Inclusivity Web Accessibility Evaluation Tools List Web Accessibility Initiative (WAI) W3C Github HTML Learn HTML — Your First Web Page (For Absolute Beginners) (4:33) CSS Learn CSS (Tutorial) — Basics + Selectors (For Absolute Beginners) (9:51) Flask/Python Harvard CS50 Week 9 SQL and Relational Databases Harvard CS50 Week 7 Web App performance testing - PageSpeed (https://pagespeed.web.dev)

Total Marks: 80

1. Identifying & Defining the Problem and solution approach (15 marks) Outcomes: SE-12-01, SE-12-06, SE-12-09	No attempt made.	Minimal attempt to define the problem. Requirements largely missing or incorrect.	Basic problem description. Limited justification, unclear feasibility, vague or incomplete requirements.	Adequate problem definition. Need is identified but justification lacks depth. Some feasibility considerations and basic requirements included.	Clear problem definition with strong justification of need. Feasibility is realistic. Functional requirements and boundaries are well described with minor omissions.	Comprehensive and precise definition of the problem. Clearly demonstrates need or opportunity, financial feasibility, detailed functional and performance requirements, accurate data structures, and clearly defined solution storyboard.
Mark Range	0	1-3	4-6	7-9	10-12	13-15
2. Research & Selection of Development Approach (10 marks) Outcomes: SE-12-01, SE-12-03	No attempt made.	Minimal research evident. Approach chosen with little explanation.	Limited research. Weak or partially incorrect justification of approach.	Basic explanation of approaches. Selected approach justified but lacks depth or clarity.	Thorough research with clear comparison. Selection is well justified with relevant reasoning.	Insightful research of Waterfall, Agile and WAgile approaches. Accurate comparison using scale, workflow and case studies. Selected approach is strongly justified for the project.
Mark Range	0	1-2	3-4	5-6	7-8	9-10
3. Project Management & Scheduling (10 marks) Outcomes: SE-12-01, SE-12-09	No attempt made.	Attempt made to organise tasks into a Gantt chart but largely ineffective.	Basic timeline with major development stages missing or unclear.	Adequate timeline; most milestones included. May include unnecessary padding or lack dependency detail.	Well-presented Gantt chart with detailed timelines, logical sequencing and clear milestones.	Comprehensive Gantt chart with fully justified scheduling and precise planning, clear dependencies, milestones and tracking. Demonstrates exemplary project management skills.
Mark Range	0	1-2	3-4	5-6	7-8	9-10
4. Producing & Implementing the Solution (25 marks) Outcomes: SE-12-02, SE-12-06, SE-12-07	No attempt made.	Minimal implementation; code largely incomplete or non-functional.	Partially functioning solution with limited documentation and inconsistent development practices.	Solution functions at a basic level. Documentation and implementation are adequate but incomplete.	Functional solution with clear documentation and correct use of tools, version control and backup procedures.	Fully functional and well-engineered solution. Algorithms clearly documented, consistent version control and backup procedures. Innovative prototyping and UI design demonstrated.
Mark Range	0	1-8	9-13	14-17	18-21	22-25

5. Testing, Optimisation & Evaluation (15 marks) Outcomes: SE-12-02, SE-12-06, SE-12-08	No attempt made.	Minimal testing evident. Little or no evaluation.	Limited testing and minimal optimisation. Evaluation largely descriptive.	Basic testing conducted. Some optimisation evident. Evaluation lacks depth.	Clear testing and optimisation processes with meaningful evaluation and response to feedback.	Comprehensive test plan implemented (Unit and Integration). Code optimisation clearly applied and evaluated. Feedback analysed and responded to with a high-quality evaluation report.
Mark Range	0	1-3	4-6	7-9	10-12	13-15
6. Documentation & Project Folio Presentation (5 marks) Outcomes: SE-12-04, SE-12-07, SE-12-09	No folio submitted.	Minimal documentation provided.	Disorganised or incomplete documentation.	Adequate documentation with some organisational or clarity issues.	Well-organised folio with clear structure and presentation.	Exceptionally well-structured, professional folio with clear headings, visuals and logical flow across all project stages.
Mark Range	0	1	2	3	4	5